

Fundamentals of System Security (COMSM0122)

Workshop 3: Proxmox Terraform Provider

Prerequisite:

- (1) Terraform installed on your local machine
- (2) **cloud-init template VM** (e.g., Ubuntu 22.04)
- (3) **Proxmox VE** is installed and accessible via its API
- (4) You've enabled the **Proxmox API** and created a user with permissions.

Overview

This workshop will introduce you to Terraform Proxmox provider - a declarative infrastructure automation tool designed for Proxmox Virtual Environment (VE). You will learn how to rapidly deploy and manage virtual machines using HashiCorp Configuration Language (HCL), enabling the creation of reproducible, isolated lab environments for penetration testing, malware analysis, and network defense simulations. The workshop emphasises modular design and best practices for infrastructure-as-code workflows.

You will first carry out a series of steps to familiarise with terraform workspace and execute associated workflows. You will then complete 4 tasks based on set workspace.

Step 1: Project Structure

- 1) Open a terminal/console
Create a folder called terraform-proxmox and change directory into the folder created:

```
mkdir terraform-proxmox && cd terraform-proxmox
```

- 2) Create the following files:

- **main.tf**
- **variables.tf**
- **outputs.tf**
- **terraform.tfvars**

```
touch main.tf variables.tf outputs.tf terraform.tfvars
```

Step 2: Configure the provider (main.tf)

Copy the following code into **main.tf**

```
terraform {
  required_providers {
    proxmox = {
      source = "Telmate/proxmox"
      version = " 3.0.2-rc04"
    }
  }
}

provider "proxmox" {
  pm_api_url      = "https://proxmox-nodeid:8006/api2/json"
  pm_api_token_id = "your-api-token" #Format: <user>@<realm>!<token_name>
  pm_api_token_secret = "your-secret-key-here"
  pm_tls_insecure  = true #Set to false if using valid TLS certs
}

resource "random_integer" "vmid" {
  count = 3
  min   = 120
  max   = 10000
}
```

proxmox-nodeid: This is the name or label used to identify a specific Proxmox VE server (node) within a cluster or standalone setup. In this workshop, this will be the IP address of Proxmox VE. For this lab, use **proxmox-nodeid = 192.168.5.11**

api2: Refers to version 2 of the Proxmox API.

json: Specifies the data format used for communication—JSON.

A **realm** is a way to define **where and how users authenticate** when accessing the system. It represents an authentication source or method. Common Realm Types in Proxmox are described in the table below.

Realm Name	Description
pam	Uses the Linux system's local user accounts (Pluggable Authentication Modules).
pve	Proxmox's built-in user database, managed directly within the Proxmox UI.
ldap/ad	Connects to external authentication systems like LDAP or Active Directory.
OpenID Connect	Allow users to log in using an external identity provider (IdP) that supports OIDC. E.g Auth0, Okta, etc

pm_api_token_id: Combines the Proxmox **user**, **realm**, and **token name**.

Example:

Assuming **realm: pve**, **user: terraform**, **Token name: mytoken**. Then the resulting token is **terraform@pve!mytoken**

pm_api_token_secret: The secret key generated when you create the token in the Proxmox UI.

pm_tls_insecure: Set to true if using self-signed certificates.

Notes: If you are using Proxmox that is locally installed on your laptop, then use the following steps to generate an access token:

- (1) Log in to the Proxmox web UI.
- (2) Go to Datacenter → Permissions → API Tokens.
- (3) Select a user (e.g., terraform@pve) and click Add.
- (4) Name the token (e.g., mytoken) and copy the secret key.
- (5) Assign appropriate permissions to the token:

Go to Datacenter → Permissions → Add

Set: (i) Path: / or a specific resource path (e.g., /vms); (ii) User or Token:

terraform@pve!mytoken; (iii) Role: Choose a role like Administrator, VM.Admin, or create a custom role; (iv) Click add.

Step 3: Paste the following proxmox_vm_qemu block into main.tf

```
resource "proxmox_vm_qemu" "web_servers" {
  count      = 3
  name       = "web-server-${random_integer.vmid[count.index].result}"
  target_node = "msccyberprox2"
  vmid       = random_integer.vmid[count.index].result
  clone      = "debian-cloud-template"

  cpu {
    cores    = 2
    sockets  = 1
  }
  memory     = 2048

  network {
    id       = 1 + count.index
    model    = "virtio"
    bridge   = "vbr0"
  }

  disk {
    slot     = "ide1"
    size     = "10G"
    type     = "disk"
    storage  = "hdd_storage"
  }
}
```

(a) Resource Overview

resource "proxmox_vm_qemu" "web_servers": Declares a Proxmox QEMU virtual machine resource named web_servers.

count = 3: Creates three identical VMs using a count loop.

name = "web-server-`{vmid}`": Each VM will be named **web-server-`[vmid-1]`**, **web-server-`[vmid-2]`**, and **web-server-`[vmid-3]`**.

(b) VM Configuration

target_node: Specifies the Proxmox node where the VMs will be created.

vmid: Assigns a unique VM ID starting from 120 (i.e., 100, 101, 102).

clone: Clones from an existing cloud-init enabled template named "**debian-cloud-template**".

cores: Allocates 2 CPU core.

memory: Allocates 1024 MB (1 GB) of RAM.

sockets: Uses 1 CPU socket.

(c) Network Configuration

model: Uses the VirtIO network adapter for better performance.

bridge: Connects the VM to the **vmbr0** network bridge on the Proxmox host.

(d) Disk Configuration

size: Allocates a **10GB** virtual disk.

type: Uses the **disk** disk interface.

storage: Stores the disk on the **hdd_storage** storage pool.

(e) Cloud-Init and IP Configuration

os_type: Enables **cloud-init**, allowing automated configuration (e.g., SSH keys, hostname).

ipconfig0: Configures the VM to obtain its IP address via **DHCP**.

Notes: If you are using Proxmox that is locally installed on your laptop, then you will have to create a cloud-init enabled server template (e.g for Ubuntu or Debian). You can achieve this by following workshop 2.

At the end of this step, you would have implemented infrastructure code that deploys 3 cloned Ubuntu VMs on proxmox-nodeld, assigned to each a unique name and VM ID, used cloud-init for provisioning, and finally connected each to the network via vmbr0 with DHCP.

Task1:

Execute the following commands after implementing the outlined steps above:

- (1) Run the command **terraform init**. Discuss the output and what happens after execution.
- (2) Run the command **terraform plan**. Discuss the output and provide reasons why it is important to preview the creation of virtual machines.
- (3) Run the command **terraform apply**. Discuss the output and what happens after execution.
- (4) Refactor the infrastructure code to use environment variables, **variables.tf** and **terraform.tfvars** files to avoid hardcoding credentials. Test your implementation by successfully executing (1) – (3)

Task 2:

In this task, your aim is to inject SSH keys directly into your VMs during creation using Terraform so that you can log in securely without passwords.

Generate an SSH key pair (`~/.ssh/id_rsa` and `~/.ssh/id_rsa.pub`) using a command-line tool called **ssh-keygen**. This tool is built into most Unix-based systems like macOS, Linux, and WSL (Windows Subsystem for Linux).

To achieve this, run the command:

```
ssh-keygen -t rsa -b 4096 -C [your_name@bristol.ac.uk]
```

- t **rsa**: Specifies the type of key (RSA)
- b **4096**: Sets the key length (4096 bits is secure)
- C: Adds a comment (usually your email)

Follow the prompts

- File location: Press Enter to accept the default (`~/.ssh/id_rsa`)
- Passphrase: Optional. Adds extra security if you want it.

You'll get two files:

(1) `~/.ssh/id_rsa.pub`

Public key - use this to inject into your VM via Terraform. Refactor your infrastructure code to inject the public key from the local file into the VMs created. Achieve this task by adding the following code to the resource `ubuntu_vm` in `main.tf`.

```
# Inject public key from local file
sshkeys = file("${path.module}/id_rsa.pub")
```

Test your code by running the command: **terraform apply**

(2) `~/.ssh/id_rsa`

Private key - keep this secret and use to SSH into the VM, Run the following command to test access:

```
ssh -i ~/.ssh/id_rsa ubuntu@<vm-ip>
```

Ensure to place your public SSH key in the same directory as your Terraform files, named `id_rsa.pub`.

Task 3:

The `outputs.tf` file is used to **define and expose values** from infrastructure deployment—These include VM IP addresses, names, or IDs—so they can be used in scripts, automation, or to verify the infrastructure that was created. As an example, the following lines of code will return as output the names of all VMs created.

```
output "vm_names" {
  description = "Names of the web server VMs"
  value       = [for vm in proxmox_vm_qemu.web_servers : vm.name]
}
```

- (a) Write an output that displays the **names**, VM IDs, and IP addresses of the VMs created. Implement an output
- (b) Run the command **terraform output**. Discuss the output and what happens after execution

Task 4:

Assume a scenario of a Proxmox-based web application architecture that is designed to satisfy the following properties:

Networking Architecture

- The infrastructure is deployed across two physical Proxmox nodes, each representing a separate fault domain to ensure high availability and redundancy.
- The network is segmented into:
 - A public bridge (**vmbr0**) that connects virtual machines to the external network for internet-facing services like reverse proxies or load balancers.
 - A private bridge (**vmbr1**) used for internal communication between backend services such as application servers and databases.
- A firewall is configured at both the host and VM level to manage traffic between public and private zones.
- DNS resolution is handled internally using a local DNS server or via integration with external DNS services like Pi-hole or Unbound.
- Outbound internet access for backend VMs is routed through a dedicated NAT VM or gateway container running on the public bridge.

Compute Resources

- Virtual Machines (VMs) are provisioned in the private network to host the application backend.
- A Proxmox cluster manages these VMs, allowing for live migration, resource balancing, and failover between nodes.
- Cloud-init templates are used to automate VM initialisation, including SSH key injection, hostname configuration, and package installation.
- A load balancer VM (e.g., HAProxy or NGINX) is deployed on the public bridge to distribute incoming traffic across backend VMs.
- Scaling is achieved manually or via scripting (e.g., using Terraform or Ansible) to clone and deploy additional VMs based on demand.

Storage and Database Services

- Static assets, backups, and logs are stored on a centralised NAS or Proxmox-integrated storage like ZFS or Ceph.
- A MySQL database VM is deployed in the private network with replication enabled across nodes to ensure high availability.
- Storage volumes are attached using Proxmox's disk management, with options for LVM, ZFS, or Ceph-backed disks.
- Snapshots and backups are scheduled using Proxmox's built-in backup tools to ensure data protection and recovery.

Security and Access Control

- Firewall rules are defined per VM and per node to control inbound and outbound traffic.

- User roles and permissions are managed through Proxmox's RBAC system to enforce least privilege access.
- SSH keys are used for secure access to VMs, with centralised key management via cloud-init or Ansible.
- Network isolation is enforced using VLANs or separate bridges, and fail2ban or similar tools are deployed for intrusion prevention.

- Generate an infrastructure diagram that describes the above scenario.
- Carry out an assessment of the scenario against the RICE Principles – describing how the scenario satisfies or does not satisfy each of the principles by completing the table below. The evaluation column should demonstrate the extent the scenario is aligned (or not aligned) with RICE.

Principle	Satisfied (Yes, No, Partially)	Evaluation
Reproducibility		
Idempotency		
Composability		
Evolvability		

- Write a short (max 100 words) summary of the scenario, describing its level of maturity and alignment with modern cloud architecture principles, suitability for iterative development, operational scalability, and enterprise-grade deployment.
- Implement the architecture specified in (a) using the Terraform Proxmox provider.

Deliverables

This is a formative assessment. Ensure one of the TAs or the course lecturer has reviewed your solutions before the end of the lab session.